

Win11 24H2 Kernel Patch Protection Analysis

Jong Hwi Park

13 June, 2025

Abstract

본 연구는 Windows Kernel Patch Protection(PatchGuard, PG)에 대한 심층 분석을 수행하고, 나아가 최초로 최신 Windows 11 24H2 버전에 구현된 PG의 내부 구조와 동작 방식을 완전 분석 및 우회한 사례를 제시한다. 본 연구는 최신 빌드 환경에서의 PG 초기화 절차, 암호화·난독화 기법, 감지·우회 로직까지 전 과정을 체계적으로 규명하였다. 이를 통해 PG의 핵심 구조를 실시간으로 추적·해체하는 기술을 확립하고, 베어메탈 환경에서 동작하는 완전한 우회 기법을 구현함으로써 보안 연구와 방어 전략 수립에 결정적인 통찰(Mitigation Insight)을 제공한다.

Contents

1	Disclaimer	3
1.1	Environment	3
2	Introduction	3
2.1	Motivation	4
2.2	Evolution of PG	4
3	Analysis	4
3.1	Init Trigger (KiFilterFiberContext)	4
3.1.1	Where is callback!?	7
3.1.2	References	8
3.2	Initialization (KiInitializePatchGuard)	10
3.2.1	PG Context	13
3.2.2	Random Padding	15
3.2.3	Critical Routines	15
3.2.4	Encryption	16
3.2.5	Entry Point	16
3.2.6	Detection Structure	17
3.2.7	VM Detection	18
3.2.8	Global PG Context	18
3.2.9	Code Section	18
3.2.10	Applies	19

3.3	Detections	21
3.3.1	BugCheck Parameter	21
3.3.2	Main FsRtlMdlReadCompleteDevEx	22
3.3.3	Sub PgTVCallbackDeferredRoutine	23
3.3.4	Sub KiSwInterruptDispatch	23
3.3.5	Sub CcBcbProfiler	24
3.3.6	Sub KiMcaDeferredRecoveryService	24
4	Approach	24
4.1	Static Analysis	24
4.1.1	Length Trick	25
4.1.2	prefetchnta Trick	26
4.1.3	xref Tracing	26
4.1.4	Obfuscation makes sense!	26
4.1.5	PG Constants	26
4.2	Dynamic Analysis	26
4.2.1	VM	27
4.2.2	Barricade	29
5	Bypass	31
5.1	Timers	31
5.1.1	KiInitializePatchGuard Method 0	32
5.1.2	CcBcbProfiler Twins	33
5.1.3	KiInitializePatchGuard Method 1, 2 DPCs	33
5.1.4	KiInitializePatchGuard Method 5	33
5.1.5	Implementation	33
5.2	MaxDataSize References	33
5.2.1	Implementation	34
5.3	Miscellaneous	34
5.3.1	KiMcaDeferredRecoveryService	34
5.3.2	System Thread & APC	34
5.3.3	Implementation	34
5.4	Barricade	34
5.4.1	Implementation	35
6	Mitigations	35
6.1	Barricade: PatchGuard is too OLD!	35
6.2	Something Else	35
7	Conclusion	36
7.1	Precautions with Demonstration	36

1 Disclaimer

이 연구는 오직 교육적 목적으로 이루어졌으며, 악용할 생각이 전혀 없음을 미리 밝힙니다.

1.1 Environment

사용된 소프트웨어 및 하드웨어는 다음과 같습니다.

- Windows 11 Home 24H2, Build 26100.4351 (Bare-Metal)
- Windows 11 Pro 23H2 (VM)
- 12th Gen Intel(R) Core(TM) i7-12700H
- VMware Workstation Player 17

2 Introduction

KPP¹는 Windows XP x64 (2005) 때부터 추가된 커널 패치 보호 매커니즘으로, PG²라고도 알려져 있습니다. 이 문서에서는 이후 PG라는 용어를 사용합니다.

PG는 이름에서도 알 수 있듯이 기본적으로 Windows NT 커널에 행해지는 모든 불법적인 패치나 변조를 보호합니다. 이는 단순히 코드 패치 뿐만 아니라 DKOM과 같은 커널 내부 구조체를 직접적으로 수정하는 것을 금합니다.

PG는 구체적으로 다음과 같은 시스템 요소를 보호하며, 더 자세한 사항은 Microsoft의 문서³에 상세히 기술되어 있습니다.

- NT 커널의 모든 함수
- 프로세서의 GDT, IDT, DR7 등 민감한 레지스터
- NT 커널의 오브젝트 및 구조체 (EPROCESS, SSDT, 여러 전역 변수)

PG는 시스템이 무결성 검사를 통과하지 못한다고 판단될 즉시 `CRITICAL_STRUCTURE_CORRUPTION` BSOD를 유발하여 악성 코드가 메모리에 상주하지 못하게 막는 것이 기본적 절차입니다.

PG는 공격자에게 지속적인 제약과 우회 시도 시 탐지 리스크를 남겨 루트킷의 활개를 효과적으로 저지하였습니다. 이러한 이유로 대부분의 악성 코드 및 게임 치트 프로그램은 커널 오브젝트에 직접적으로 데이터를 기록하지 않거나, NT 커널의 핵심 함수에 대한 후킹을 회피하는 등 PG를 철저히 고려하여 설계됩니다.

¹Kernel Patch Protection

²PatchGuard

³<https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/bug-check-0x109---critical-structure-corruption>

2.1 Motivation

공격자들은 대부분 PG를 우회할 수 없다고 생각하기 때문에, PG와의 정면 충돌을 회피합니다. 하지만 일부 고수준의 해커는 PG를 우회하고 심각한 위협을 일으키는 APT 기법을 개발합니다.

본 연구의 목적은 단순한 PG 분석이나 우회 기술의 구현에 있지 않고, 고수준의 공격자들이 활용하는 내부 매커니즘을 정밀하게 이해한 뒤 **방어적 통찰**을 제공하는 데 핵심 의도가 있습니다.

분석을 진행한 저 또한 PG 우회자들과 동등한 기술력을 지니기 위해 프로젝트를 진행하게 되었습니다.

2.2 Evolution of PG

PG는 지금까지 끊임없이 진화해왔습니다. 초기 Windows XP 및 Vista의 PG는 BSOD를 유발하는 함수인 `KeBugCheckEx`를 직접 후킹하여 우회할 수 있는 단순한 수준이었으나, Windows 7부터는 함수 진입 전에 미리 체크섬을 검증하는 방식으로 이러한 우회 기법을 막았습니다. 이후 Windows 8 이상에서는 PG 자체의 체크섬을 지속적으로 검증하는 등 더욱 정교해지면서 우회가 어려워졌으며, Windows 11에서는 가상화 기술의 본격적인 도입과 함께 HyperGuard(HG)가 등장하여 한층 더 견고한 보호 기법이 구현되었습니다.

지금까지 다양한 연구자들이 PG 대한 분석 자료를 공개해왔지만, 대부분 수 년 전 작성된 것으로 최신 Windows 11 버전에 대한 정보는 거의 존재하지 않습니다. 이에 따라, 저는 작성일 기준으로 가장 최신 빌드인 Windows 11 24H2를 대상으로 PG 분석을 직접 진행하였습니다.

3 Analysis

과거부터 PG 분석이 활발히 이루어져 왔습니다. 저는 그런 과거 연구 자료를 참고해 가며 기본적인 정보를 숙지한 뒤 분석을 시작하였습니다.

PG는 분석가들에게 혼란을 주기 위해 아예 다른 이름으로 심볼화되어 있거나, (e.g. `KiFilterFiberContext`) 심볼이 없습니다. 나중에 알아가겠지만 정상적인 루틴 내에 숨겨진 PG 루틴이 있는 경우 또한 있습니다.

3.1 Init Trigger (`KiFilterFiberContext`)

기본적으로 PG는 `ntoskrnl.exe`가 초기화될 때 같이 초기화가 이루어집니다. 여러 NT 모듈이 비밀스럽게 PG Init을 호출합니다.

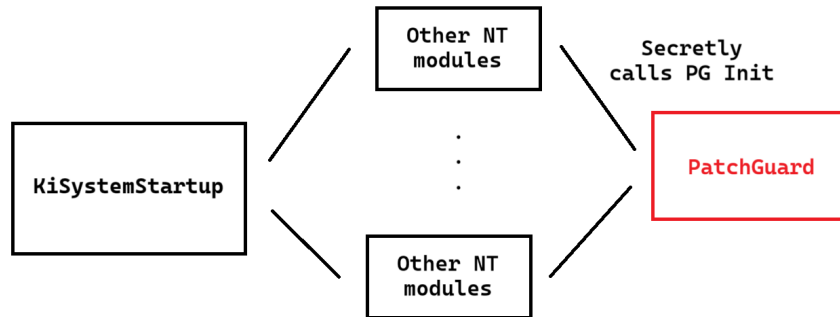


Figure 1: PatchGuard Initialization

PG를 초기화하는 함수는 `KiInitializePatchGuard`로 알려져 있는데, 해당 함수를 호출하는 다른 함수 중 가장 중심적인 함수는 `KiFilterFiberContext`라고 볼 수 있습니다. 즉, 직접적인 초기화는 `KiInitializePatchGuard`에서 수행하지만, 이를 준비하고 트리거하는 'Stub' 함수 중 가장 중요한 함수가 `KiFilterFiberContext`입니다. 아무래도 PG 분석에 있어 매우 중요한 함수 중 하나이죠. 이 외에도 다른 함수가 있으나, 가장 자주 호출되고 눈여겨볼만한 함수는 해당 함수입니다. 따라서 먼저 `KiFilterFiberContext` 함수를 조사해보겠습니다.

지금부터 제시하는 코드들은 모두 컨텍스트가 매우 크기 때문에, 의사 코드로 변형하여 제공할 것입니다. 매우 큰 컨텍스트를 옮기는 과정에서 의도하지 않은 손실이나 비약이 있어 이해에 어려움이 있을 수 있습니다. 제가 작성한 의사 코드는 언어적으로 이해하여 전체적인 흐름을 파악하는 용도로 사용해 주시기 바랍니다.

Algorithm 1: KiFilterFiberContext

Data: FIBER_CONTEXT_PARAM fiberParam

Result: BOOLEAN result

```
1 AntiDebug;
2 if MaxDataSize not initialized then
3   if PsIntegrityCheck then
4     create callback "542875F90F9B47F497B64BA219CACF69";
5     notify callback;
6   end
7 end
8 r1 ← rand(0, 12);
9 r2 ← rand(0, 5);
10 r3 ← rand(0, 9);
11 result ← KiInitializePatchGuard(r1, r2, r3 ≤ 5, fiberParam, 1);
12 if result then
13   if r1 ≤ 5 then
14     r4 ← rand(0, 12);
15     r5 ← rand(0, 5) (≠ r2);
16     result ← KiInitializePatchGuard(r4, r5, r3 ≤ 5, fiberParam,
17                                     0);
18   end
19   if result then
20     result ← KiInitializePatchGuard(0, 7, 1, 0, Unknown);
21   end
22 AntiDebug;
23 ClearTails;
24 return result
```

보면, KiInitializePatchGuard 함수를 랜덤한 파라미터로 호출합니다. 즉, 해당 KiFilterFiberContext 함수는 KiInitializePatchGuard를 호출하는 발사대 역할을 하는 함수임을 알 수 있고, KiInitializePatchGuard 함수는 여러 번 호출될 수 있음을 시사합니다.

ClearTails의 경우, 초기화가 끝나고 런타임에 PG 관련된 컨텍스트를 메모리에서 찾을 수 없게 만드는 코드입니다.

fiberParam은 NULL일 수 있습니다. 앞으로 서술하겠지만, NULL인 경우가 대부분입니다.

MaxDataSize의 경우 앞으로 설명할 PG 글로벌 포인터입니다. 나중에 제대로 다룹니다.

rand 함수는 앞으로도 그렇겠지만, $[a, b)$ 가 아닌 $[a, b]$ 구간에 대해 랜덤을 생성

하는 것을 참고하시기 바랍니다.

3.1.1 Where is callback!?

콜백과 관련하여, 실제로 코드를 보면 다음과 같이 되어 있습니다.

```
1 ObjectAttributes.Length = 48;
2 ObjectAttributes.ObjectName = (PUNICODE_STRING)L"TV";
3 ObjectAttributes.RootDirectory = 0LL;
4 ObjectAttributes.Attributes = 64;
5 *(_OWORD *)&ObjectAttributes.SecurityDescriptor = 0LL;
6 if ( ExCreateCallback(/*CallbackObject*/ &CallbackObject, /*
   ObjectAttributes*/ &ObjectAttributes, /*Create*/ 0, /*
   AllowMultipleCallbacks*/ 0) >= 0 )
7 {
8     ExNotifyCallback(/*CallbackObject*/ CallbackObject, /*
       Argument1*/ PgTVCallbackDeferredRoutine, /*Argument2
       */ &__24);
9     ObfDereferenceObject(/*Object*/ CallbackObject);
10    if ( __24 )
11        __2c = 1;
12    ExInitializeNPagedLookasideList(/*Lookaside*/ &
      stru_140E0EF80, /*Allocate*/ 0LL, /*Free*/ 0LL, /*
      Flags*/ 0x200u, /*Size*/ 0xB38uLL, /*Tag*/ 0
      x746E494Bu, /*Depth*/ 0);
13 }
```

많은 연구자들이 ObjectName이 "TV"로 되어 있어 콜백 이름을 'TV 콜백'이라 칭하지만, 이것은 디컴파일러의 오류로 인한 오해입니다. 자료형이 UNICODE_STRING 이기 때문에 실제로 콜백 이름은 다릅니다.

실제 이름은 "\Callback\542875F90F9B47F497B64BA219CACF69" 입니다. 별개로 'TV 콜백'이라는 단어가 더 보편적이기 때문에, 본 문서에서는 이후로 'TV 콜백'이라는 단어를 사용합니다.

커널 콜백은 대부분 ExCreateCallback, ExRegisterCallback, ExNotifyCallback 순서의 시나리오를 따릅니다. ExCreateCallback에서 생성된 콜백을 ObjectName 을 통해 ExRegisterCallback에서 등록한 뒤, ExNotifyCallback을 통해 등록된 모든 콜백에 대해 함수를 호출합니다. ExCreateCallback과 ExRegisterCallback 의 순서는 큰 영향을 끼치지 않습니다.

하지만 코드를 보면 ExRegisterCallback은 어디에도 보이지 않습니다. 즉, 어디선가 이미 ExRegisterCallback을 사용한 것으로 보입니다. 사실 어디서 호출하였는지는 중요하지 않습니다. ExNotifyCallback을 통해 호출되는 루틴, 즉 PgTVCallbackDeferredRoutine이 중요합니다. (이것이 실제 검사에 사용되는 루

틴이기 때문입니다.) 이 함수에 대해서는 나중에 다뤄보겠습니다.

3.1.2 References

KiFilterFiberContext 함수를 호출하는 여러 모듈들에 대해 알아보겠습니다. 전에도 언급했듯, 여러 모듈에서 기발한 방법을 통해 호출합니다.

3.1.2.1 KeInitAmd64SpecificState

이 함수로 인해 항상 확정적으로 KiFilterFiberContext가 호출됩니다. 이름만 보면 언뜻 AMD64 CPU에 대해 특정한 작업을 수행하는 것 같습니다. 실제로 이 함수는 PipInitializeCoreDriversAndElam라는 정상적인 함수에 자연스럽게 끼어 있습니다.

Algorithm 2: KeInitAmd64SpecificState

Data: None

Result: None

```
1 if KdDebuggerNotPresent and KdPitchDebugger then
2   | KiFilterFiberContext(0);
3 end
```

하지만 실제로는 KiFilterFiberContext만을 호출하는 함수입니다. 이 함수는 디버깅 환경이 아닌 경우에 대해 Exception Handler를 통해 ZeroDivisionError를 일부러 발생시킨 뒤 핸들링하는 것으로, 언뜻 보면 아무것도 하지 않는 루틴으로 보여 분석가들을 혼란에 빠뜨립니다.

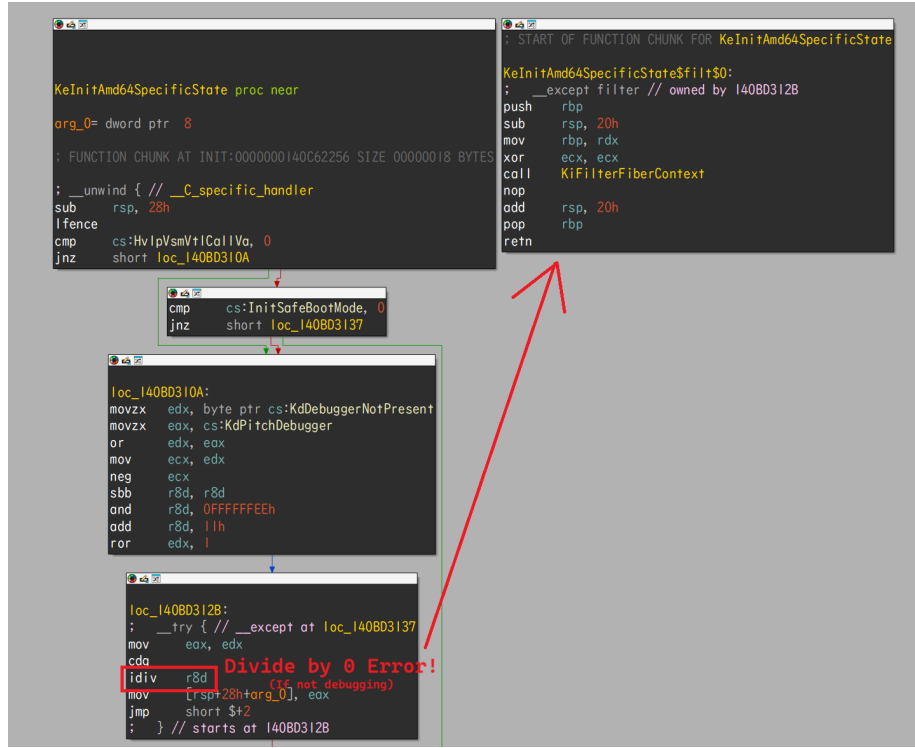


Figure 2: KeInitAmd64SpecificState handling

여기서 언뜻 추리할 수 있는 사실은, PG는 디버그 환경일 경우 초기화를 하지 않는다는 것입니다. 이는 분석가들을 방해하려는 목적의 설계입니다.

3.1.2.2 ExpLicenseWatchInitWorker

이 함수는 ExpWatchProductTypeInitialization 함수에서 호출됩니다. ExpWatchProductTypeInitialization 함수는 Microsoft 라이선스를 검증하는 함수들이 많은 곳으로, 이 함수 역시 같은 부류인 함수인 척 하며 자연스럽게 PG를 초기화합니다.

Algorithm 3: ExpLicenseWatchInitWorker

Data: None

Result: None

- 1 fiberParam = KPRCB.HalReserved[6];
 - 2 if rand(0, 99) < 4 and KdDebuggerNotPresent and KdPitchDebugger
then
 - 3 | KiFilterFiberContext(fiberParam);
 - 4 end
-

이 함수의 경우 4%의 확률로 fiberParam을 동반하여 KiFilterFiberContext를 호출합니다. fiberParam의 경우 미리 초기화된 CPU의 PRCB.HalReserved[6] 멤버를 이용합니다. 물론 초기화가 끝나면 해당 멤버의 흔적은 사라져 전달된 fiberParam을 확인할 수 없습니다.

fiberParam의 자료형인 FIBER_CONTEXT_PARAM은 다음과 같습니다.

```

1 struct FIBER_CONTEXT_PARAM {
2     UCHAR Code[4]; // prefetchw byte ptr [rcx]; ret
3     DWORD Unk0;
4     PVOID PsCreateSystemThreadPtr; // Used in Method 3
5     PVOID PsCreateSystemThreadDeferredStub; // Used in
        Method 3
6     PVOID KiBalanceSetManagerPeriodicDpcPtr; // Used in
        Method 5
7 }

```

이 구조체는 나중에 KiInitializePatchGuard 함수에서 중요한 구조체로 자리합니다.

3.1.2.3 Extra KiVerifyXcpt15

이 함수는 KiFilterFiberContext를 거치지 않고 KiInitializePatchGuard 루틴을 호출합니다. 이 함수는 본래 정상적인 역할을 수행하지만, 디버깅 상태가 아닐 경우 KeInitAmd64SpecificState 함수처럼 Exception Handler를 통해 PG 초기화 루틴을 호출합니다.

해당 함수는 KiVerifyXcptRoutines 함수 배열 안의 멤버이며 KiVerifyScopesExecute 함수가 해당 배열 안의 함수를 모두 호출하는 것으로 트리거됩니다.

Algorithm 4: KiVerifyXcpt15

Data: Unknown

Result: Unknown

```

1 ...
2 if KdDebuggerNotPresent and KdPitchDebugger then
3     | KiInitializePatchGuard(rand(0, 12), 1, 2, 0, 0);
4 end
5 ...
6 return Unknown

```

3.2 Initialization (KiInitializePatchGuard)

KiInitializePatchGuard 함수는 본래 심볼이 없는 함수로, KiFilterFiberContext 및 KiVerifyXcpt15 함수에서 KeExpandKernelStackAndCallout 함수를 동반하

여 호출되는 함수입니다.

해당 함수는 `ntoskrnl.exe`의 함수 중 가장 길이가 긴 함수입니다. 실제로 디컴파일을 시도하면 약 32000줄의 코드가 등장합니다. 물론 이것은 난독화, 인라인 블록으로 인한 영향이 포함되어 있지만, 기본적으로 수행하는 일이 많은 함수입니다.

Algorithm 5: KiInitializePatchGuard (1/2)

```
Data: DWORD dpcIndex, DWORD method, DWORD vUnk0,  
        FIBER_CONTEXT_PARAM fiberParam, BOOLEAN largeCheck  
Result: BOOLEAN result  
1 if "TESTSIGNING" and "DISABLE_INTEGRITY_CHECK" in Boot Options  
  and code patches are allowed then  
2   | return False  
3 end  
4 if method = 5 and fiberParam is NULL then  
5   | method = 0;  
6 end  
7 crArr = CriticalRoutines;  
8 PG_Context = MemoryAllocate(Size: 0x100000 + 0xAE8 +  $\alpha$ );  
9 PG_Context.AddRandomPadding();  
10 PG_Context.Fill(CmpAppendDllSection's Bytecode);  
11 for  $f$  in NT Functions (that is used in this function) do  
12   | PG_Context.Fill( $f$ );  
    | // It is used to disturb analyzers. It works like VM.  
    |   e.g. It stores ExAllocatePool2 in PG_Context+32, so  
    |   they call like (PG_Context+32)(...)  
13 end  
14 PG_Context.Fill(crArr's PTE);  
    // It is used to recover when triggering BSOD, a mitigation  
    for 'hooking KeBugCheckEx' method  
15 for  $f$  in NT Functions do  
16   | PG_Context.AddDetection( $f$ );  
17 end  
18 if largeCheck then  
19   | for  $f$  in NT Functions (More) do  
20   | | PG_Context.AddDetection( $f$ );  
21   | end  
22 end  
23 PG_Context.AddDetection(IDT);  
24 PG_Context.AddDetection(GDT);  
25 PG_context.AddStructureDetections();
```

Algorithm 6: KiInitializePatchGuard (2/2)

```
1 PG_Context.SetPerProcessorState();
2 PG_Context.SetCodeSectionInContext();
3 PG_Context.SetEntrypointProperly();
4 PG_Context.FinalizeCodeStuff();
  // It is used to set code section in PG context self.
5 if method = 3 then
6   pgThread = PG_Context.InitializePgThread();
7   PsCreateSystemThread(pgThread);
8 end
9 if method = 4 then
10  pgApc = PG_Context.InitializePgApc();
11 end
12 if method = 5 then
13  PG_Context.InitializeHook(fiberParam);
14 end
15 if method = 7 then
16  InitializeGlobalContext(MaxDataSize, PG_Context);
17 end
18 PG_Context.finalize();
  // The final routine is to verify itself, the stored
  // routines, etc
19 PG_Context.EncryptSelf();
  // And they encrypt self, ready to launch.
20 pgDpc = PgDpcRoutines[dpcIndex];
21 switch method do
22   case 0 do
23     KeSetCoalescableTimer(pgDpc);
24   end
25   case 1 do
26     KPRCB.AcpiReserved = pgDpc;
27   end
28   case 2 do
29     KPRCB.HalReserved[7] = pgDpc;
30   end
31   case 4 do
32     KeInsertQueueApc(pgApc);
33   end
34   case 3, 5, 7 do
35     Unknown;
36   end
37 end
38 return True
```

저는 모든 루틴을 하나하나 정독하면서 분석하지 않고, 다양한 기법을 사용하여 필요한 부분만 분석했기 때문에 많은 부분이 생략되었습니다. 즉, 제가 조사했을 때 발견한, 실제 검사에 영향이 미치는 것만을 의사 코드에 포함하였습니다. 이 방대한 루틴을 이와 같이 매우 짧은 알고리즘으로 이해하기에는 무리가 있습니다. 서브섹션에서 자세히 설명하겠습니다.

KiInitializePatchGuard 루틴을 크게 두 부분으로 나누게 되면, '초기화 부분'과 '적용 부분'으로 나눌 수 있습니다. 현재 환경 기준으로 약 26000줄까지가 초기화 부분, 다음부터는 적용 부분입니다.

당연히 중요한 부분은 적용 부분입니다. 적용 부분을 자세히 파악할 수만 있다면, 아주 쉽게 우회할 수 있는 경우가 대부분입니다.

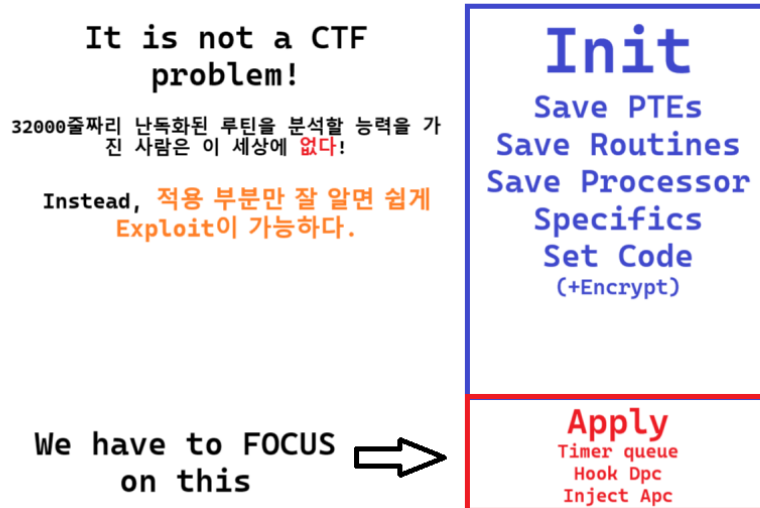


Figure 3: Important insight about KiInitializePatchGuard

따라서 적용 부분에 힘을 실어 설명할 예정입니다. 제시된 알고리즘도는 서브섹션을 모두 읽고 다시 와서 보시는 것을 강력히 추천드립니다.

3.2.1 PG Context

PG Context는 PG와 관련된 모든 변수, 코드, 검사 루틴 목록 등 다양한 컨텍스트를 저장하는 거대한 구조체입니다. 현재 빌드 24H2에서 측정된 크기는 차그마치 2792 (0xAE8) 바이트입니다.

중요한 멤버를 몇 가지 알아보겠습니다.

```
1 struct PG_Context {
2     UCHAR CmpAppendDllSectionCode[sizeof(CmpAppendDllSection
3     )];
4     UnkSz0 vUnk0;
5     struct _FnVm {
6         PVOID strnicmpPtr;
7         PVOID KiFreezeDataTableEntryPtr;
8         // ...
9     } FnVm;
10    UnkSz1 vUnk1;
11    struct _DetectionRoutine {
12        UINT64 Type;
13        PVOID TargetRoutine;
14        DWORD DetectionSize;
15        DWORD Hash;
16        UINT64 Desire0, Desire1, Desire2;
17    } *DetectionRoutines;
18    UnkSz2 vUnk2;
19    UnkSz3 MethodSpecific;
20    UnkSz4 vUnk3;
21    // ===== After 0xAE8 Size =====
22    struct _PgCodeSection {
23        UnkSz5 vUnk4;
24        UCHAR InitKDBGSection[sizeof(INITKDBG)];
25        UnkSz6 vUnk5;
26    } PgCodeSection;
27 }
```

- CmpAppendDllSectionCode: CmpAppendDllSection 함수의 바이트코드를 저장하는 곳입니다. 후에 진입점으로 활용됩니다.
- FnVm: KiInitializePatchGuard 함수에서 사용하는 NT 함수들의 포인터를 저장합니다. 이는 분석가들을 방해하기 위한 것으로 보입니다. 이것이 선언된 이후 거의 모든 함수는 간접 호출로 호출됩니다.
- DetectionRoutines: 검사하는 함수들을 담은 동적 배열입니다. largeCheck 파라미터 (5번째)에 의해 크기가 달라집니다.
- MethodSpecific: 각 method 파라미터 (2번째)에 대해 개별적으로 저장되는 파라미터입니다.
- PgCodeSection: PG는 중단점, 후킹 등을 막기 위해 코드 영역을 컨텍스트에 따로 저장합니다. 크기가 0x100000 추가된 이유가 이 때문입니다.
- InitKDBGSection: ntoskrnl.exe의 INITKDBG 섹션을 통째로 갖다 둔 곳입니다.

3.2.2 Random Padding

PG Context를 포함한 PG와 연관된 구조체는 랜덤한 수의 랜덤한 바이트 (총 2047, 0x7FF개)를 구조체의 앞, 뒤에 추가합니다. 이로 인해 메모리에서 PG 관련 구조체 찾기가 더욱 어려워집니다.

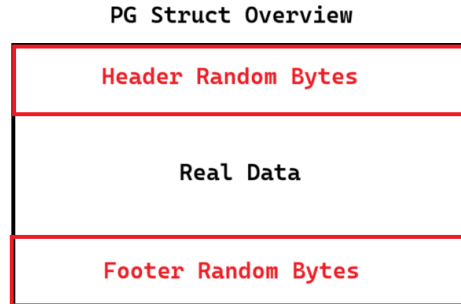


Figure 4: PG Struct Overview

하지만 이는 좋은 점 또한 있는데, 바로 16 바이트 정렬이 깨진다는 것입니다. 다른 NT 구조체의 경우 대부분 16 바이트로 정렬되어 있는 반면에 PG 구조체 포인터의 경우 16 바이트 정렬이 안 되어 있어, 이것이 PG 구조체인지 쉽게 확인이 가능해집니다.

3.2.3 Critical Routines

PG가 특별히 중요하게 여기는 루틴들이 있는데, 이것을 저는 Critical Routine 이라고 부르기로 결정하였습니다. 목록은 다음과 같습니다.

```
CriticalRoutines dq 0 ; DATA XREF: sub_140BD5620+E33 ↑ w
; sub_140BD5620+E8E ↑ r ...
dq offset KeBugCheckEx
dq offset KeBugCheck2
dq offset KiBugCheckDebugBreak
dq offset KiDebugTrapOrFault
dq offset DbgBreakPointWithStatus
dq offset RtlCaptureContext
dq offset KeQueryCurrentStackInformation
dq offset KiSaveProcessorControlState
dq offset memmove
dq offset IoSaveBugCheckProgress
dq offset KeIsEmptyAffinityEx
dq offset VfNotifyVerifierOfEvent
dq offset _guard_check_icall_no_overrides
dq offset KeGuardDispatchICall
```

Figure 5: Critical Routines

가장 첫 번째 원소는 현재 NULL이지만, 런타임에 HaliHaltSystem으로 초기화 됨

니다.

이 Critical Routine들은 변형될 시 안전한 시스템 종료에 영향이 가는 함수들이기 때문에, 따로 PTE까지 저장하여 둡니다. 이는 나중에 검사에 걸려 BSOD를 유발해야 할 시 복구됩니다.

3.2.4 Encryption

PG는 자신을 암호화하여 저장합니다. 이는 초기화가 거의 완료되었을 때 실행하며, PG를 런칭하기 전에 암호화를 완료합니다. 기본적으로 XOR 키로 암호화되며, 이중 암호화 구조를 띄고 있습니다.

첫 번째 암호화는 CmpAppendDllSection, KiDpcDispatch 에서 해제되는 암호화입니다. 이 암호화는 코드 섹션을 포함한 전 구간을 암호화하며 PG Context 자체를 메모리에서 못 찾게 하는 역할을 합니다.

두 번째 암호화는 각종 검사 루틴 (대표적으로 FsRtlMdlReadCompleteDevEx)에서 해제되는 데이터 암호화입니다. 이 암호화는 설령 분석자가 코드 섹션에 어찌저찌 접근을 하였다 하더라도 데이터에 접근하지 못하게 막는 역할을 합니다.

3.2.5 Entry Point

모든 PG Context의 진입점은 세 함수 중 하나로 직결됩니다: CmpAppendDllSection, KiDpcDispatch, KiTimerDispatch. 우선 dpcIndex 파라미터 (1번째)에 따라 PgDpcRoutines[dpcIndex]로 pgDpc가 결정됩니다. PgDpcRoutines 배열은 다음과 같습니다.

```
1 PVOID PgDpcRoutines[13] = {
2     CmpEnableLazyFlushDpcRoutine,
3     CmpLazyFlushDpcRoutine,
4     ExpTimeRefreshDpcRoutine,
5     ExpTimeZoneDpcRoutine,
6     ExpCenturyDpcRoutine,
7     ExpTimerDpcRoutine,
8     IopTimerDispatch,
9     IopIrpStackProfilerDpcRoutine,
10    KiBalanceSetManagerDeferredRoutine,
11    PopThermalZoneDpc,
12    KiDpcDispatch, // Or KiTimerDispatch
13    KiDpcDispatch, // Or KiTimerDispatch
14    KiDpcDispatch // Or KiTimerDispatch
15 };
```

이들은 모두 PG의 초기 진입점이 되는 DPC 루틴들로, 여타 다른 정상적인 DPC들과 같이 DeferredContext가 존재합니다.

이 중 처음 열 개의 루틴에 대해 자세히 살펴보면, 모두 KiCustomAccessRoutineN

과 관련이 있다는 것을 확인할 수 있습니다. 이 DPC 루틴들은 본래 정상적인 역할을 수행하다가, DeferredContext로 Canonical하지 않은 주소가 전달되면 KiCustomAccessRoutineN을 호출합니다. 이 KiCustomAccessRoutineN들은 내부적으로 KiCustomRecurseRoutineN을 각각 호출합니다.

이제 KiCustomRecurseRoutineN을 계속해서 재귀하며 돌다가 DeferredContext 값을 포인터로 역참조를 시도합니다. 이때 DeferredContext는 유효한 주소가 아니기 때문에 예외가 발생하고, 숨겨진 핸들러로 이동하게 되어 결국 CmpAppendDllSection을 호출하게 됩니다.

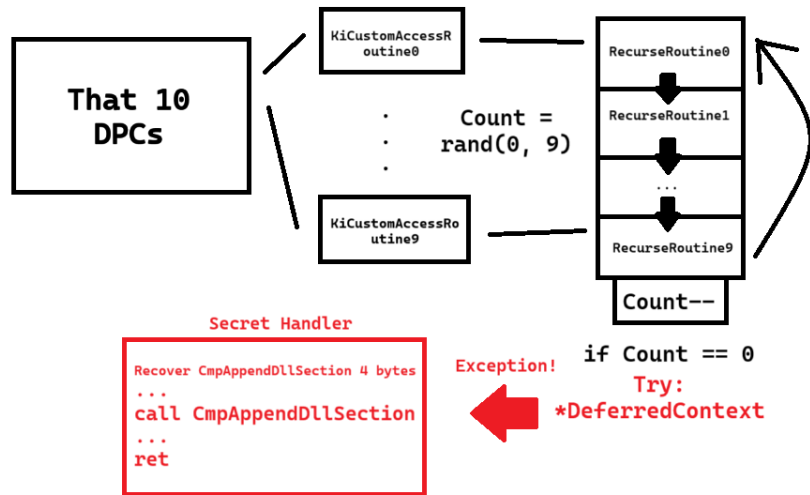


Figure 6: CmpAppendDllSection Calling Scenario

CmpAppendDllSection 함수는 이제 자신의 컨텍스트를 복호화한 뒤 검사 루틴으로 들어갑니다. KiDpcDispatch 함수는 저러한 복잡한 과정 없이 직접적으로 자신의 컨텍스트를 복호화하고 검사 루틴을 호출합니다.

KiTimerDispatch 함수를 엔트리 포인트로 가진 PG Context는 KiDpcDispatch와 마찬가지로 복잡한 과정 없이 직접적으로 호출하지만, 자신의 코드를 복호화하는 첫번째 복호화 과정을 수행하지 않습니다.

3.2.6 Detection Structure

PG는 NT 함수의 무결성을 검사할 때 특정한 구조체를 이용합니다. 이는 전에 언급한 구조체 _DetectionRoutine입니다.

```

1 struct _DetectionRoutine {
2     UINT64 Type;
3     PVOID TargetRoutine;

```

```

4     DWORD DetectionSize;
5     DWORD Hash;
6     UINT64 Desire0, Desire1, Desire2;
7 } *DetectionRoutines;

```

- **Type:** 버그 체크의 유형입니다. CRITICAL_STRUCTURE_CORRUPTION KeBugCheckEx의 네 번째 파라미터와 일치합니다.
- **TargetRoutine:** 타겟 함수의 주소입니다. 굳이 함수가 될 필요는 없고, 특정 커널 오브젝트여도 됩니다. (e.g. IDT, SSDT, etc)
- **DetectionSize:** 검사할 크기입니다.
- **Hash:** 검사의 결과가 정상적일 경우 도출되는 해시 값입니다.
- **Desire0, Desire1, Desire2:** Type에 따른 특정 동작을 제어합니다. 가령 IDT를 검사할 경우 일부러 Exception을 내보고, EPROCESS 구조체를 검사할 때는 Flink와 Blink를 검사하는 것을 정의하는 필드입니다. 검사 루틴에서 이 필드들을 이용해 특정 동작을 수행합니다.

3.2.7 VM Detection

저는 테스트 중 상당수를 VM에서 진행하였는데, VM에서 PG의 동작이 변경되는 것을 포착하였습니다. 이는 Windows 11에 들어서 새로 생긴 기능으로, 하이퍼바이저가 존재하는 환경에서 PG의 움직임이 크게 달라집니다. 대표적으로 KeExitRetpoline 함수를 통해 특정 VM 환경에서 검사를 아예 스킵하는 행위가 있습니다.

이 외에도 INITKDBG 섹션 페이지에 KiErrata로 시작하는 함수들은 모두 하이퍼바이저 모듈을 감지하는 비정규적 방법을 사용하고 있습니다.

자세히 분석해 보지 않아 얼마나 많은 하이퍼바이저 감지 기술이 있는지는 모르겠습니다. 하지만 분명한 것은, VM 환경은 PG를 분석하기에 좋지 않은 환경이라는 것입니다.

3.2.8 Global PG Context

method가 7인 경우 MaxDataSize 전역 변수에 PG Context가 초기화 됩니다. 이때 항상 코드를 복호화하는 1단계 암호화가 빠져 있으며, 그에 따라 KiTimerDispatch 함수를 진입점으로 항상 이용합니다. MaxDataSize는 추후 KiSwInterruptDispatch 함수에서 검사할 때 이용되며, TV 콜백에 전달되는 PgTVCallbackDeferredRoutine에서도 쓰입니다.

3.2.9 Code Section

PG는 관련된 PG 함수, 예를 들어 FsRtlMdlReadCompleteDevEx, CmpAppendDllSection와 같은 함수를 따로 컨텍스트 코드 영역에 저장하고, 해당 PG 함수를 수행할 때

ntoskrnl.exe가 아닌 PG Context에서 실행합니다. 이는 분석가들이 PG 함수에 후킹이나 BP를 거는 것을 방지합니다.

3.2.10 Applies

이렇게 초기화된 PG Context가 실제로 적용이 되는 부분을 살펴보겠습니다. method에 따라 PG가 실제로 적용되는 양상이 달라집니다.

3.2.10.1 Method 0

method가 0일 경우 dpcIndex에 의해 결정된 pgDpc를 DPC로 사용하는 타이머를 초기화한 뒤, KeSetCoalescableTimer 함수로 타이머를 큐합니다.

이때 주목해야 할 점은 큐 되는 KTIMER 오브젝트의 Period 멤버가 0이고, DueTime 멤버에 약 2분 정도의 값이 들어있습니다. 'PG의 경우 계속해서 검사를 수행해야 할 텐데 왜 Period 값이 아닌 DueTime을 쓰는가' 하는 의문이 들었습니다. 검사 루틴을 확인해 보면 모두 검사를 마친 뒤 다시 KeSetCoalescableTimer 함수로 타이머를 큐 하는 것을 볼 수 있습니다. 즉, 추가적인 보안을 위해 DueTime을 선택하고 검사가 끝날 때마다 새로운 타이머를 넣는 것을 택한 것입니다.

3.2.10.2 Method 1

method가 1일 경우 결정된 pgDpc를 바탕으로 KPRCB.AcpiReserved 멤버에 해당 DPC를 삽입합니다. 해당 DPC가 어떻게 호출되는지 모르겠지만, 결과적으로 호출되는 것을 확인했습니다.

3.2.10.3 Method 2

method가 2일 경우 method 1과 거의 유사하지만 KPRCB.HalReserved[7] 멤버에 해당 DPC를 삽입합니다. method 1과 마찬가지로 호출되는 것을 확인했습니다.

3.2.10.4 Method 3

method가 3일 경우 pgDpc는 사용되지 않습니다. 대신 fiberParam 내의 PsCreateSystemThreadPtr 함수 포인터로 쓰레드를 만든 뒤 PsCreateSystemThreadDeferredStub 함수를 삽입합니다.

이 외에도 6개의 다른 함수가 삽입되는데, 이 함수들은 호출되지 않았습니다.

해당 함수 내에는 검사 간격을 확보하기 위해 다음 세 개의 함수를 사용합니다: KeDelayExecutionThread, KeWaitForSingleObject or KeWaitForMultipleObjects.

언뜻 보면 fiberParam 내의 함수 포인터가 필요한 것 같지만, 이것이 진정 fiberParam을 필요로 하는지는 의문입니다. fiberParam이 없는 경우에도 PsCreateSystemThread 함수를 호출하는 것이 포착되었습니다. 실제로도 정상적으로 PG Context가 초기화된 모습 또한 볼 수 있었습니다. 자세히 알아보진 않았으나, 다른 방법으로 호출하는 것 같습니다.

3.2.10.5 Method 4

method가 4일 경우 이미 있는 랜덤한 시스템 쓰레드에 KeInsertQueueApc 함수로 간격이 약 2분가량 되는 APC를 주입합니다. 이 APC는 Method 3과 마찬가지로 프로시저 실행에 지연을 일으키는 함수를 사용하여 검사 간격을 확보합니다.

3.2.10.6 Method 5

method가 5인 경우, fiberParam이 NULL일 경우에는 method가 0으로 폴백됩니다. 즉, 결론적으로 method가 5일 확률이 굉장히 적습니다. 이 메소드는 fiberParam의 멤버 KiBalanceSetManagerPeriodicDpcPtr를 이용해 KiBalanceSetManagerPeriodicDpc 전역 변수를 후킹합니다.

내부적으로 카운터를 두어 원래 정상적인 루틴을 수행하다가 카운터가 0이 될 경우 pgDpc를 큐하는 방식으로 진행됩니다. 카운터는 초기에 랜덤한 값으로 초기화 되고, 0이 될 때마다 pgDpc를 호출하고 다시 랜덤한 값으로 초기화됩니다.

Method 5

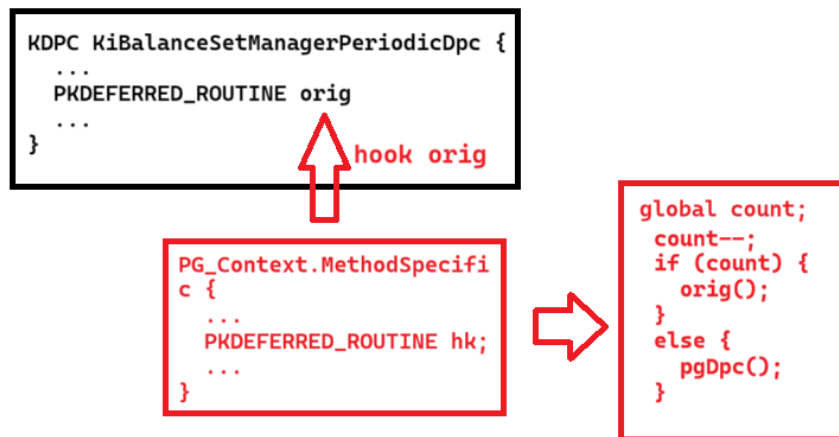


Figure 7: Method 5 Visualization

3.2.10.7 Method 7

method가 6인 경우는 존재하지 않습니다. 이유는 저도 잘 모르겠습니다.

method가 7인 경우 아무것도 하지 않습니다. KiInterruptThunk 혹은 KiMachineCheckControl 함수로 DPC를 초기화하는 모습까지 확인되었으나, 해당 DPC를 큐 하지 않고

KiInitializePatchGuard가 종료됩니다.

실제로 method가 7인 경우를 배제하고 우회 코드를 작성하였는데, 전혀 문제 없이 우회가 정상적으로 되는 모습을 볼 수 있었습니다.

method 7은 시스템 부팅 시 적어도 한 번은 호출됩니다. KiFilterFiberContext에서 마지막으로 항상 고정된 파라미터를 두기 때문입니다.

마지막으로 method 7은 글로벌 PG Context 포인터인 MaxDataSize 전역 변수를 초기화합니다.

3.3 Detections

이제 PG가 어떤 식으로 시스템 손상을 감지하는가를 알아보겠습니다. _DetectionRoutine.Type에 따라 검사 방법이 천차만별입니다. _DetectionRoutine.DesireN 멤버를 이용해 각 Type에 대해 개별적으로 행해지는 행위들을 정의할 수 있습니다.

3.3.1 BugCheck Parameter

다음과 같은 인라인 블록들이 다수 존재합니다.

```
1  *(_QWORD *) (v2 + 2336) = v2 - 0x5C5FC0A76E374B18LL;
2  *(_QWORD *) (v2 + 2344) = (char *)v38 - 0x4C48B4211BBACBEBLL;
3  v68 = *v38;
4  *(_QWORD *) (v2 + 2360) = v67;
5  v69 = *(_DWORD *) (v2 + 2520);
6  *(_QWORD *) (v2 + 2352) = v68;
7  *(_DWORD *) (v2 + 2328) = 1;
8  if ( (v69 & 0x20000000) == 0 && *(_DWORD *) (v2 + 2524) & 0
9      x2000000) != 0 && (v69 & 1) != 0 )
10 {
11     v70 = *(unsigned int *) (v2 + 2676);
12     v71 = *(_QWORD *) (v2 + 2104);
13     v72 = *(_QWORD *) (v2 + 2680);
14     v73 = (_QWORD *) (v70 + v2);
15     v74 = v70 + v2 + 8 * ((unsigned __int64)(unsigned int)
16         *(_DWORD *) (v2 + 2052) - v70) >> 3);
17     while ( v73 != (_QWORD *)v74 )
18     {
19         *v73 ^= v72;
20         v72 = ((v71 ^ *v73++) + __ROR8__(v72, v72 & 0x3F)) ^
21             0xEFA;
22     }
23     *(_DWORD *) (v2 + 2524) &= ~0x200000u;
24     if ( v72 != *(_QWORD *) (v2 + 2688) )
25     {
26         v75 = *(_DWORD *) (v2 + 2052);
27         v76 = *(_QWORD *) (v2 + 1416);
```

```

25     *(_QWORD *)v76 = v2;
26     *(_DWORD *)(v76 + 16) = v75;
27     if ( !*(_DWORD *)(v2 + 2328) )
28         *(_QWORD *)(*(_QWORD *)(v2 + 1416) + 24LL) = v72
29             ^ *(_QWORD *)(v2 + 2688);
30     sub_140BCC384(a1: v2, a2: 0LL, a3: v72, a4: 256LL);
31 }

```

이는 검사를 통과하지 못했을 때 호출되는 공통된 루틴으로, KeBugCheckEx에 들어가는 버그 체크 파라미터를 정의합니다. 즉, 예약된 파라미터의 정체를 알 수 있습니다.

- Parameter1: 검사를 수행한 PG Context 주소
- Parameter2: 검사를 통과하지 못한 _DetectionRoutine 구조체의 주소
- Parameter3: 손상된 데이터의 주소
- Parameter4: 손상된 데이터의 Type

Parameter1, Parameter2는 각각 상수 0x5C5FC0A76E374B18와 0x4C48B4211BBACBEB와 뺄셈을 수행하여, 분석가들이 접근하는 것을 일차적으로 저지합니다.

Parameter4에 관련하여, 공식적으로 문서화되지 않은 265와 같은 Type이 존재하는데, 이는 PG Context의 손상과 관련된 경우가 대부분입니다.

해당 인라인 블록은 검사 루틴을 정적 분석할 때 아주 도움이 됩니다. 어느 부분으로부터 검사가 실패하였는지 쉽게 알 수 있게 되기 때문입니다.

3.3.2 Main FsRtlMdlReadCompleteDevEx

모든 Type에 대한 검사를 한꺼번에 진행하는 거대한 함수입니다. 함수는 우선 데이터 부분을 복호화한 뒤, Microsoft BugCheck 문서에 정의된 모든 Type에 대해 검사를 수행합니다. 마지막으로, 정리 루틴을 수행하는데, 여기서는 만료된 타이머를 다시 큐 하는 등의 작업을 수행합니다.

검사를 통과하지 못했을 경우 CriticalRoutines에 대한 PTE를 복구하고 SdbpCheckDl1 함수를 호출합니다. 이 함수는 KeBugCheckEx를 호출하는 래퍼 함수이지만, 호출하기 전에 스택을 정리합니다. 이는 메모리 덤프를 분석하려는 분석가들을 막는 방법입니다.

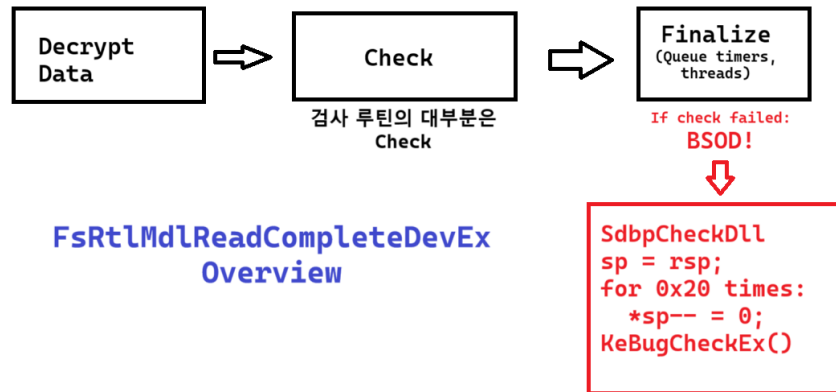


Figure 8: FsRtlMdlReadCompleteDevEx Overview

3.3.2.1 Nb PgDetectionN

이 외에도 존재하는 심볼이 없는 검사 함수가 몇 가지 있습니다. 저는 이들에게 간단하게 PgDetectionN 이라고 이름지어 주었습니다. 이들은 모두 FsRtlMdlReadCompleteDevEx 와 하는 행위가 거의 같지만, 차이점은 검사 Type 범위가 작습니다. 이들은 VM이 실행중일 경우에만 동작하는 것을 확인했고, Bare-Metal에서 동작하는 것은 포착하지 못했습니다.

이들을 자세히 분석하지 않은 이유는 VM에서만 동작하기 때문입니다. 우리의 목표는 Bare-Metal에서 우회를 구현하는 것이지, VM에서 구현하는 것이 아니기 때문입니다.

3.3.3 Sub PgTVCallbackDeferredRoutine

PgTVCallbackDeferredRoutine는 TV 콜백에 파라미터로 전달되는 함수로 FsRtlMdlReadCompleteDevEx 와 동일한 역할을 수행하지만 마지막에 타이머, 쓰레드를 다시 큐하는 작업이 없습니다. 이 콜백이 어떻게 재호출되는지는 의문입니다.

해당 함수는 Method 7에서 초기화된 MaxDataSize PG 글로벌 컨텍스트를 사용합니다.

3.3.4 Sub KiSwInterruptDispatch

최근에 추가된 검사 루틴인 것 같습니다. 해당 함수는 KiSwInterrupt IDT 함수에서 호출되는 루틴입니다. 함수의 이름과는 정 반대로, PG 검사를 수행합니다.

이 또한 마찬가지로 Method 7에서 초기화된 MaxDataSize를 사용합니다.

3.3.5 Sub CcBcbProfiler

이 함수는 PG Context와는 독자적으로 운행되는 루틴입니다. 정확히 말하자면, KiInitializePatchGuard와 관련이 없습니다.

이 함수는 CcInitializeBcbProfiler 에서 KeSetCoalescableTimer 함수로 타이머가 초기화되어 약 2분마다 랜덤한 NT 커널의 한 함수를 조사합니다. 쌍둥이 함수 CcBcbProfiler2 (원래 심볼이 없습니다.) 가 존재하는데, 이 함수와 협동하여 검사 루틴이 끝나면 서로를 다시 KeSetCoalescableTimer로 큐 하여 재실행합니다.

다른 점은, BSOD를 SdbpCheckDll 함수가 아닌 CcAdjustBcbDepth로 호출합니다. 두 함수의 역할은 동일합니다.

3.3.6 Sub KiMcaDeferredRecoveryService

이 외에도 독자적으로 운행되는 루틴이 여러 가지 있습니다. (e.g. PspProcessDelete, KiInitializeUserApc) 이 함수들은 검사를 독립적으로 수행하는 것이 아닌, 정상적인 역할을 하며 검사도 같이 하는, 역할을 수행하기 전에 가볍게 검사하는 성격이 강합니다. 이 함수들은 공통적으로 BSOD를 발생시킬 때 KiMcaDeferredRecoveryService 함수를 호출합니다.

KiMcaDeferredRecoveryService 함수는 SdbpCheckDll, CcAdjustBcbDepth와 같이 스택을 정리하고 KeBugCheckEx를 호출하는 함수입니다.

4 Approach

PG를 분석하기 위해 다양한 분석법을 사용하였습니다. 이전까지 서술한 분석 결과는 지금부터 제시할 방법들을 이용해 구한 것입니다.

4.1 Static Analysis

실제로 저는 모든 분석 시간 중 가장 많은 시간을 이 정적 분석에 소모하였습니다. 다행인 점은 PG 함수가 가상화가 되어있지 않고 난독화만 적용되어 있기 때문에, 노력한다면 코드 분석을 수행할 수 있다는 점입니다.

분석을 진행하다 보면 인라인으로 추정되는 부분들을 다수 발견할 수 있습니다.

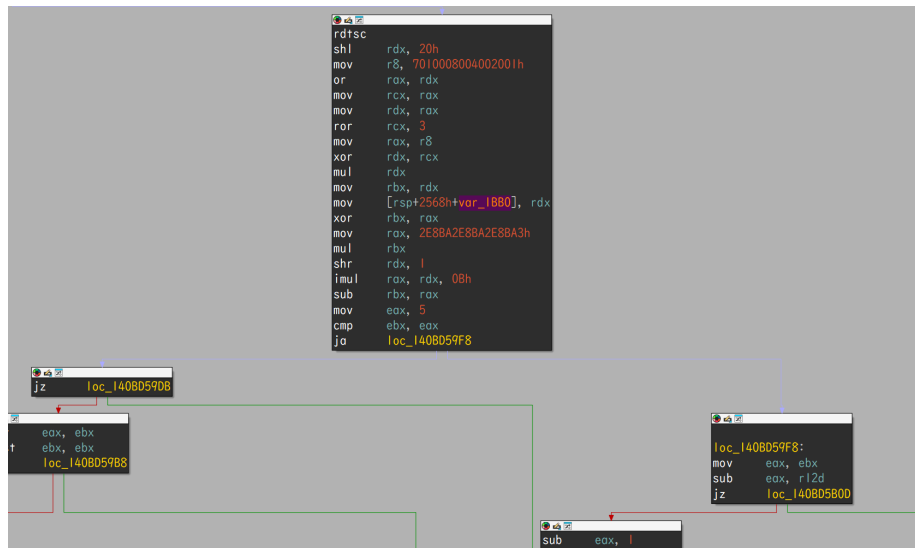


Figure 9: Pool Allocation

예를 들어 다음과 같은 `rdtsc` 명령으로 시작하여 분기가 많이 존재하는 인라인 블록은 풀 할당을 할 때 풀 태그로 식별할 수 없게, 랜덤한 알파벳 네 글자를 생성할 때 사용되는 인라인 어셈블리입니다.

분석을 할 때는 이러한 인라인 블록들을 파악하고 빠르게 넘기는 데에 집중했습니다. 정적 분석을 하는 데에는 대단한 도구나 트릭을 사용하지 않고 정직하게 시간을 투자하였습니다.

하지만 역설적이게도 가장 성과가 없는 방법이었습니다.

4.1.1 Length Trick

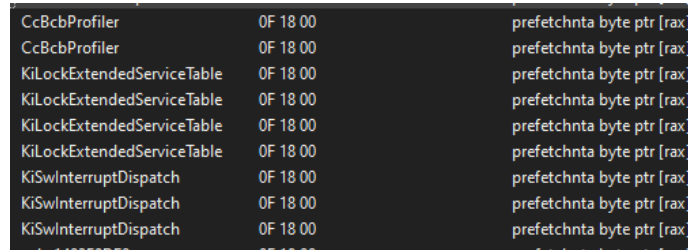
전에 난독화가 되어 있다고 말했는데, 난독화 기법의 특성에 따라 코드 길이가 불어나게 됩니다. 이 점에 착안하여 `ntoskrnl.exe`에서 코드 길이가 긴 순서대로 정렬하여 보면 실제로 다수의 PG 루틴들이 포진되어 있는 것을 볼 수 있습니다.

SEAccessCheckByType	.text	00...	000020C2	00...	00...
CmpDoParseKey	PAGE	00...	00002D36	00...	00...
NtQueryInformationToken	PAGE	00...	00002F95	00...	00...
NtSetInformationJobObject	PAGE	00...	00003196	00...	00...
sub_140981100	PAGE	00...	000032AB	00...	00...
ExAllocateHeapPool	.text	00...	00003D03	00...	00...
NtQueryInformationProcess	PAGE	00...	00004CA9	00...	00...
PropertyEval	PAGE	00...	00004E4A	00...	00...
NtSetInformationProcess	PAGE	00...	000052E4	00...	00...
ExQuerySystemInformation	PAGE	00...	000061D7	00...	00...
LZ4HC_compress_generic_dict...	.text	00...	00007BD8	00...	00...
PgTvcallbackDeferredRoutine	.text	00...	0000EB20	00...	00...
FsRtlMdlReadCompleteDevEx	INITKDBG	00...	00012797	00...	00...
KlInitializePatchGuard	INIT	00...	000275CA	00...	00...

Figure 10: Capture PG routines by function length

4.1.2 prefetchnta Trick

PG와 관련된 중요한 루틴들은 대개 `prefetchnta byte ptr [rax]` 를 포함하게 됩니다. 해당 오프코드를 왜 호출하는지는 자세히 알 수 없지만, 검사를 할 때 성능을 향상시킬 수 있기 때문이라고 추측할 수 있습니다.



CcBcbProfiler	0F 18 00	prefetchnta byte ptr [rax]
CcBcbProfiler	0F 18 00	prefetchnta byte ptr [rax]
KiLockExtendedServiceTable	0F 18 00	prefetchnta byte ptr [rax]
KiLockExtendedServiceTable	0F 18 00	prefetchnta byte ptr [rax]
KiLockExtendedServiceTable	0F 18 00	prefetchnta byte ptr [rax]
KiLockExtendedServiceTable	0F 18 00	prefetchnta byte ptr [rax]
KiSwInterruptDispatch	0F 18 00	prefetchnta byte ptr [rax]
KiSwInterruptDispatch	0F 18 00	prefetchnta byte ptr [rax]
KiSwInterruptDispatch	0F 18 00	prefetchnta byte ptr [rax]

Figure 11: PG Functions using prefetchnta

따라서 PG를 분석할 때 시그니처 `0F 18 00`을 검색하면 효과적으로 발췌하여 분석할 수 있습니다.⁴

4.1.3 xref Tracing

NT 관련 함수들은 간접 호출을 거의 쓰지 않아서, 어떤 함수의 Cross Reference를 조사한다면 해당 함수를 호출하는 부모 함수가 그대로 노출될 가능성이 큼니다.

4.1.4 Obfuscation makes sense!

어떤 함수를 보았을 때 '이것이 PG 함수인가?'에 대한 답변을 도출할 수 있게 만드는 것은 역설적이게도 난독화입니다. PG 함수들은 난독화를 위해 잘 사용하지 않는 `rdtsc`, `rol`, `ror` 등의 어셈블리어를 자주 사용하기 때문에, 이 점을 착안하여 함수를 본다면 보다 쉽게 PG 함수인지 파악이 가능합니다.

4.1.5 PG Constants

BSOD를 유발할 때 사용되는 두 가지 상수가 존재합니다: `0x5C5FC0A76E374B18`, `0x4C48B4211BBACBEB`. 이들은 버그 체크 코드 `0x109`와는 달리 암호화가 되어 있지 않은 평문이기 때문에, 시그니처 검색을 수행할 수 있습니다.

이 중에서 두 번째 상수 `0x4C48B4211BBACBEB`는 NULL일 경우가 존재하기 때문에, 저는 `0x5C5FC0A76E374B18`를 사용하였습니다.

4.2 Dynamic Analysis

PG의 경우 동적 분석이 아주 힘든 프로그램에 속합니다. Windows에 특화된 디버거인 WinDbg⁵는 사용할 수 없습니다. WinDbg를 사용하려면 Windows가 디버그

⁴Credit: daax

⁵<https://learn.microsoft.com/ko-kr/windows-hardware/drivers/debugger/>

모드가 켜져 있어야 하고, PG는 디버그 모드의 경우 활성화되지 않기 때문입니다.

혹자는 'Kd 관련된 플래그를 원상태로 복구하면 안 되는 것인가?' 라고 생각할 수 있습니다. 하지만 디버깅에 관련된 플래그가 몇 개인지도 파악이 쉽지 않으며 KdDisableDebugger, cli-sti를 비롯한 안티 디버깅 루틴이 존재했기 때문에, 이것들을 모두 고려하기엔 소모가 너무 심합니다.

4.2.1 VM

하이퍼바이저(VM)을 외곽 디버거로 삼으면, 게스트 OS 입장에서 인터럽트, 디버그 레지스터와 같은 흔적이 전혀 남지 않은 채로 완전히 Out-of-band 디버깅이 가능합니다.

핵심은 VM-exit 트랩을 디버거 이벤트로 변환하는 것입니다. 인텔의 VT-x의 경우 게스트 OS가 Debug Register를 옮기는 연산이 하이퍼바이저에 먼저 전달되기 때문에 (VM-exit이 트리거됩니다.), 이것을 하이재킹하는 것이 구체적인 방법입니다.

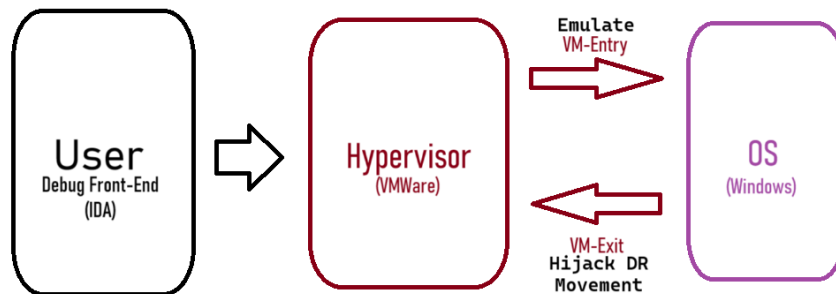


Figure 12: VM Debugging

VMware⁶는 해당 기능에 대한 지원이 있습니다. VM의 설정 파일(.vmx)에 다음과 같은 항목을 추가하면 됩니다.

```
1 debugStub.listen.guest64 = "TRUE"
2 monitor.debugOnStartGuest64 = "TRUE"
3 debugStub.port.guest64 = "1234"
```

⁶<https://www.vmware.com/>

이렇게 작성하고 실행한다면 `debugStub.port.guest64` 포트에 `gdb`⁷가 열리게 되며, IDA⁸ 디버그 프론트 엔드로 해당 VM에 Remote GDB Debugger로 연결하면 정상적으로 VM을 디버깅할 수 있습니다.

VM을 켜면 분석했듯이 Control Flow가 달라져서 PG가 정상적으로 작동을 하지 않지만, 초기화 루틴(`KiInitializePatchGuard`)을 분석하기에 용이합니다.

초기화 단계에서 호출되는 함수들, 특히 간접 호출을 파악하는데 용이하며, PG Context의 특정 부분에 R/W Trace를 걸어서 어떤 값이 어떤 시점에 복사가 되는지 직관적으로 확인할 수 있습니다. 실제로 타이머와 관련된 대부분의 루틴 동작을 이 방법을 통해 유추해 내었습니다.

디버깅은 `KiSystemStartup` 함수가 아니라 EFI 로딩 단계에서부터 시작되므로, 정확한 진입점을 찾기 어렵습니다. 또한 이 시점에서는 `ntoskrnl.exe`가 메모리에 할당되는 주소와 같은 필수적인 정보들을 확인할 수 없습니다.

저희 목표는 NT 커널이 초기화될 때 같이 초기화되는 PG를 분석하는 것이 목적이기 때문에, 이 문제는 굉장히 치명적입니다.

이 어려움을 해결하기 위해 저는 `KUSER_SHARED_DATA` 구조체에 주목하였습니다. `KUSER_SHARED_DATA`는 시스템의 고정된 주소(`FFFF'F780'0000'0000`)를 사용하며, 시스템 초기화에 사용되는 함수입니다. 실제로 `KiInitializeKernel` 함수에 다음과 같이 `KUSER_SHARED_DATA+280`을 레퍼런스하는 것을 확인할 수 있었습니다.

```

008 {
009     v10 = __readcr3();
010     __writecr3(v10);
011 }
012 }
013 KiSetPageAttributesTable();
014 if ( MEMORY[0xFFFFF78000000280] )
015     v11 = BugCheckParameter1 | 0x80000000;
016 else
017     v11 = BugCheckParameter1 & 0xFFFFFFFF3FFFFFFuLL | 0x40000000;
018 BugCheckParameter1 = v11;
019 v12 = __readcr4();
020 __writecr4(v12 | 0x18);
021 if ( KiFlushPcid )
022 {

```

Figure 13: `KiInitializeKernel` using `KUSER_SHARED_DATA`

이제 간단합니다. 우리는 `FFFF'F780'0000'0280`에 Hardware R/W Breakpoint를 설정할 수 있고, 실제로 실행해보면 `ntoskrnl.exe`가 메모리에 할당된 후에 BP가 트리거되는 것을 볼 수 있습니다.

여기까지의 모든 작업을 매 디버깅 시에 손수 하기엔 무리가 있으므로 자동화된

⁷<https://sourceware.org/gdb/>

⁸<https://hex-rays.com/>

작업을 위해 IDAPython Script를 작성하였습니다. 코드는 Git 저장소 Research 폴더⁹에서 확인할 수 있습니다.

여기까지 완료하게 되면 초기화 루틴을 자세히 분석할 수 있습니다. 이 외에도 OS가 실행 중일 때 언제든지 OS를 Suspend하여 분석할 수도 있습니다.

4.2.2 Barricade

이 방법이 정말 분석에 치명적인 역할을 하였습니다. 이 방법은 따로 정형화된 기법이 아니고, PG의 특성을 분석하여 얻어낸 방법론입니다.

PG Context는 자신을 암호화하여 저장하며, CmpAppendDllSection 혹은 KiDpcDispatch를 진입점으로 사용하는 컨텍스트는 함수를 통해 진입하고 자신을 복호화합니다. 실행 가능한 어셈블리 코드이니 실행 권한(X)가 있어야 하는 것은 자명합니다. 또한, 해당 페이지 안의 컨텍스트를 복호화해야 하니, 읽기/쓰기 권한(R/W)이 있어야 합니다. 따라서 두 함수를 진입점으로 하는 패치 가드의 진입점은 항상 RWX 권한을 가지고 있습니다.

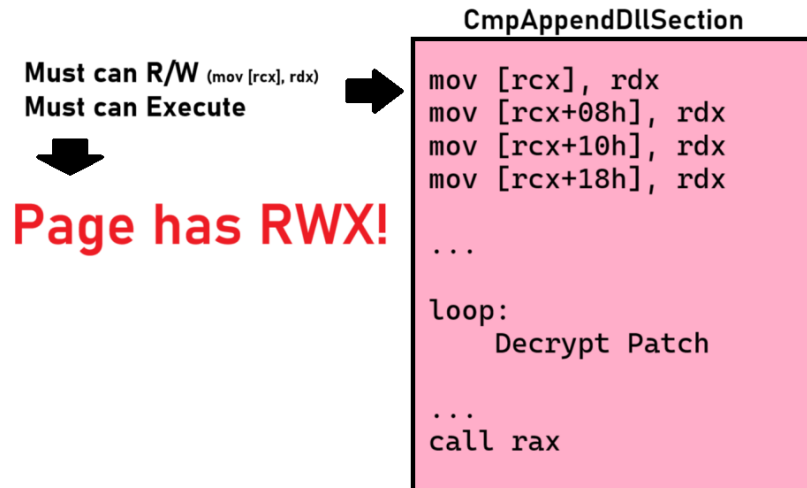


Figure 14: Proof of CmpAppendDllSection has RWX

여기서 'PG Context 진입점은 항상 RWX인가?' 추론을 할 수 있고 KiTimerDispatch 함수를 포함해서 실제로 세 함수의 페이지의 PTE를 살펴본 결과 모두 RWX 권한을 가지고 있음이 확인되었습니다. KiTimerDispatch 함수는 RWX 권한이 없어도 되는데 RWX였다는 점이 놀라웠습니다.

⁹<http://cl.wane.im:25819/Wane/kurasagi/tree/7d8c832dea17e0b35971ddde0c6afa0bba315dd9/research>

이것이 중요한 이유가 있습니다. 마이크로소프트의 드라이버 서명 정책을 살펴 보면, 서명받은 드라이버는 **RWX** 섹션이 단 1바이트라도 존재하면 안 됩니다.¹⁰ Inf2Cat 또한 NonPagedPool (= NonPagedPoolExecute) 풀이 존재할 경우 테스트를 통과시키지 않게 두고 있습니다.

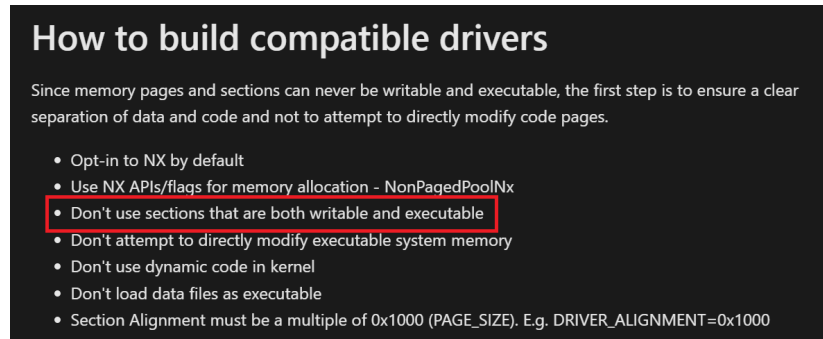


Figure 15: Microsoft Driver Enforcement to should not have RWX section

즉, 정상적인 Windows 11 시스템에서는 RWX 섹션을 가진 드라이버 코드가 없어야 합니다. 여기서 **RWX** 섹션을 가진 모듈은 **PG 뿐이다**라는 아이디어를 도출해 낼 수 있습니다. (ntoskrnl.exe의 몇몇 모듈은 RWX 섹션을 사용하지만, 매우 적은 수입니다.) 이는 나중에 구현에서 오버헤드, 오류가 적은 이유와 연관됩니다.

이제 모든 페이지를 순회하며 RWX 섹션 중 X 권한을 제거합니다. 이렇게 되면 PG 진입점에서 처음으로 코드를 실행할 때 X 권한이 없기 때문에 **페이지 폴트 (#PF) 예외가 발생하게 됩니다.**

#PF를 담당하는 함수는 IDT의 KiPageFault 함수입니다. 이 중에서도 위와 같이 X 권한이 없을 때 실행을 시도했을 경우엔 MmAccessFault 함수로 Redirect 됩니다.

이제 MmAccessFault 함수를 후킹하여 PG에 대한 예외만 추가적으로 핸들링할 수 있습니다. 저는 분석할 때 일부러 스택 정보를 포함하여 BSOD를 발생시켜 분석을 더욱 수월하게 만들었습니다.

¹⁰<https://learn.microsoft.com/en-us/windows-hardware/test/hlk/testref/driver-compatibility-with-device-guard>

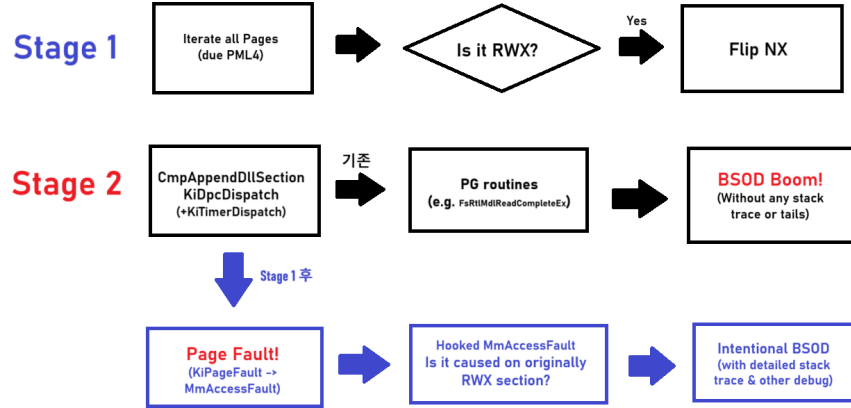


Figure 16: Algorithm of Barricade Debug method

PG를 분석하기 어려운 이유는 첫째로 컨텍스트가 암호화되어 있어서 PG 루틴이 실행 중이 아닐 때 컨텍스트를 발견하기 어렵기 때문이고, 둘째로 루틴이 어디서 어떻게 호출되는지 알 수 없기 때문에 어려운 것인데, 이 방법은 두 가지 문제점을 동시에 해소할 수 있는 강력한 기법입니다.

저는 'PG 루틴을 진입하기도 전에 #PF를 발생시켜 막는다'라는 아이디어가 마치 진입을 막는 바리케이트(Barricade)와 닮아있어, 이 방법을 Barricade라고 명명하였습니다.¹¹

5 Bypass

저는 어떻게 우회를 해야 하는가에 대한 고찰을 하였습니다. 물론 가장 쉬운 방법 몇 가지를 소개하자면, 살펴본 것처럼 시스템 자체를 디버그 모드로 부팅하거나, VT-x를 이용해 EPT 후킹을 하여 아주 쉽게 우회할 수 있지만 목표는 베어메탈로 일반적인 시스템 런타임에 PG를 우회하는 것이기 때문에, 해당 방법론들을 사용할 수 없습니다.

KiInitializePatchGuard 함수의 흐름을 변경할 수 없다는 점이 크게 작용합니다. 하지만 언급했듯이 적용 부분과 어떤 함수가 어떤 경위로 호출되는지 제대로 알고 있다면, 각 함수에 대해 우회 시나리오를 구성할 수 있습니다..

5.1 Timers

현재까지 발견된 Timer 관련 적용 및 검사 루틴은 다음과 같습니다. 다음 모듈들을 모두 종료시켜야 합니다.

¹¹Credit: can1357, PgC: Garbage Collecting PG away

- KiInitializePatchGuard Method 0
- CcBcbProfiler & CcBcbProfiler2
- DPC: KiInitializePatchGuard Method 1
- DPC: KiInitializePatchGuard Method 2
- Hook: KiInitializePatchGuard Method 5

method 1, 2 또한 결국에는 타이머와 연관이 있기 때문에 타이머에 포함하였습니다.

타이머는 `PRCB.TimerTable`에 연결 리스트 형태로 저장되는 것이 일반적입니다. 따라서, 모든 프로세서를 순회하며 `PRCB.TimerTable`을 스캔하는 식으로 모든 레지스터된 타이머를 조사할 수 있습니다.

문제는 타이머 엔트리 안의 DPC 루틴을 정의하는 `Dpc` 멤버는 `KiWaitAlways`, `KiWaitNever`와 암호화되어 저장되었다는 것입니다. 이는 `KeSetTimer` 함수의 초반에서 발견할 수 있었습니다.

```

1 v4 = (unsigned __int64)Timer ^ _byteswap_uint64(KiWaitAlways
    ^ (unsigned __int64)Dpc);
2 v42 = Dpc;
3 v5 = (unsigned int)KiWaitNever;
4 v6 = DueTime;
5 v7 = (_KDPC *) (KiWaitNever ^ __ROR8__(v4, KiWaitNever));

```

발견한 내용을 바탕으로 복호화하는 함수를 작성하면 다음과 같습니다.

```

1 PKDPC wsbp::Timer::GetDecryptedDpc(PKTIMER pTimer) {
2     return (PKDPC)(*gl::RtVar::KiWaitAlwaysPtr ^
3         _byteswap_uint64(
4             (UINT64)pTimer ^ _rotl64(
5                 (UINT64)pTimer->Dpc ^ *gl::RtVar::KiWaitNeverPtr
6                 , \
7                 (UCHAR)*gl::RtVar::KiWaitNeverPtr \
8             )));
9 }

```

5.1.1 KiInitializePatchGuard Method 0

Method 0에서 `pgDpc`로 전달되는 `DeferredContext` 값은 Canonical한 주소가 아니라고 언급한 적이 있습니다. (섹션 3.2.5)

따라서, `DeferredContext`가 Canonical한 주소가 아닌 경우를 체크하여, 발견될

경우 Dpc 자체를 NULL로 만들어서 해결할 수 있습니다.

MSDN에도 Dpc가 NULL이어도 된다고 나와 있습니다.¹²

DeferredRoutine을 확인하지 않는 이유는, 첫 번째로 원래 그 함수는 정상적인 역할을 하기 때문에 판별할 수 없다는 것이고, 두 번째로 KiDpcDispatch 및 KiTimerDispatch 함수는 언급했듯이 PG Context로 옮겨지기 때문에 판별하기 어렵습니다.

5.1.2 CcBcbProfiler Twins

이것의 경우는 더 간단합니다. 왜냐하면 DeferredRoutine으로 확정지을 수 있기 때문입니다. DeferredRoutine이 CcBcbProfiler 혹은 CcBcbProfiler2인지 확인하고 똑같이 Dpc를 NULL로 만들면 해결됩니다.

5.1.3 KiInitializePatchGuard Method 1, 2 DPCs

Method 1의 경우 KPRCB.AcpiReserved, Method 2의 경우 KPRCB.HalReserved[7]에 DPC를 저장하는 것을 알고 있습니다.

이들의 본래 값은 NULL 입니다. NULL이 아닌 값이 목격될 경우 그저 다시 NULL로 변경해주면 됩니다.

5.1.4 KiInitializePatchGuard Method 5

KiBalanceSetManagerPeriodicDpc 변수를 원래대로 복구하면 됩니다. 원래 DeferredRoutine은 KiBalanceSetManagerDeferredRoutine 입니다.

5.1.5 Implementation

전체 구현 코드는 Git 저장소에서 확인할 수 있습니다.¹³

5.2 MaxDataSize References

Method 7에서 초기화되는 PG 글로벌 컨텍스트 MaxDataSize에 접근, 사용하는 루틴은 다음과 같습니다.

- KiSwInterruptDispatch
- PgTVCallbackDeferredRoutine

이 경우엔 기본적으로 MaxDataSize를 NULL으로 맞추는 것으로 해결이 가능합니다. 하지만 문제가 있습니다. 다른 루틴들은 MaxDataSize를 역참조하기 전에 MaxDataSize가 NULL인지 확인하지만, KiSwInterruptDispatch 함수는 확인하지 않고 역참조를 시도하여 BSOD를 유발합니다.

¹²<https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-kessettimer>

¹³<http://cl.wane.im:25819/Wane/kurasagi/blob/master/kurasagi/Module/Timer.cpp>

이를 반영하여 `KiSwInterruptDispatch` 함수 자체를 패치하여 검사를 스킵할 수 있습니다. 이 방법은 PG가 이것 이외의 활성화된 루틴이 없을 때 가능합니다.

5.2.1 Implementation

전체 구현 코드는 Git 저장소에서 확인할 수 있습니다.¹⁴

5.3 Miscellaneous

기타 루틴들은 다음과 같습니다.

- `KiMcaDeferredRecoveryService`
- System Thread: `KiInitializePatchGuard` Method 3
- APC: `KiInitializePatchGuard` Method 4

5.3.1 KiMcaDeferredRecoveryService

해당 함수를 패치하여 `KeBugCheckEx`가 동작하지 않게 하면 됩니다. `SdbpCheckDll`과 `CcAdjustBcbDepth` 함수를 호출하는 루틴은 `JUMPOUT` 명령을 통해서 만일 루틴이 실패했을 경우 트랩에 걸리게 해 놓았지만 `KiMcaDeferredRecoveryService` 함수를 호출하는 루틴은 그런 것이 없습니다. 이는 하나의 취약점으로 작용합니다.

5.3.2 System Thread & APC

쓰레드나 APC는 내부적으로 `KeDelayExecutionThread`, `KiWaitForSingleObject`, `KiWaitForMultipleObjects`를 호출하여 프로시저를 지연시킨다고 언급한 바 있습니다. (섹션 3.2.10.4)

시스템 쓰레드를 순회하며 콜 스택을 조사합니다. 리턴 주소에 세 함수 중 하나라도 있다면 `KeDelayExecutionThread`를 `DueTime`을 아주 큰 수로 두고 호출하여 종료시키면 됩니다.

5.3.3 Implementation

전체 구현 코드는 Git 저장소에서 확인할 수 있습니다.¹⁵

5.4 Barricade

바리케이트 방법은 바리케이트 디버깅 방법 (섹션 4.2.2)과 전까지 나왔던 검사 우회 방법을 융합한 총체적 검사 우회 기법입니다.

¹⁴<http://cl.wane.im:25819/Wane/kurasagi/blob/master/kurasagi/Module/Context7.c>

pp

¹⁵<http://cl.wane.im:25819/Wane/kurasagi/blob/master/kurasagi/Module/Misc.cpp>

이 방법은 PG 진입점에서 실행 시 #PF를 발생시켜 후킹된 `MmAccessFault` 함수를 통해 추가적인 조사를 감행하여 PG를 식별하는 방법입니다.

문제는 `MmAccessFault` 함수는 여러 곳에서 호출된다는 것입니다. 발생한 예외가 PG 루틴인 것인지 판별하기 원래라면 어려웠을 것이고, 오버헤드가 엄청났을 것입니다. 하지만 우리가 증명한 사실에 따르면 간단한 RWX 권한 체크만으로 식별 정확도를 매우 크게 늘리고 오버헤드를 감축할 수 있다는 것이 알려졌습니다.

5.4.1 Implementation

PG를 우회하는 전체 구현 코드는 Git 저장소에서 확인할 수 있습니다.¹⁶ 설명은 간단하지만 실제로 구현하는 것은 굉장히 어렵습니다. 코드를 한 번씩 보시는 것을 강력히 권해 드립니다.

6 Mitigations

지금까지의 우회 기법을 고찰하며, 어디서 방어할 수 있을 것인지 고찰해 보도록 하겠습니다.

6.1 Barricade: PatchGuard is too OLD!

현재 가장 치명적인 방법인 Barricade인데, 저로서는 Barricade 방법을 막는 방법이 생각나지 않습니다. RWX 권한을 사용하지 않는다면 Signature Scan에 뚫릴 것이기 때문입니다.

생각의 전환을 할 필요가 있습니다. 우리는 커널에서 커널 검사를 하지 말고 더욱 위 권한을 얻어 검사하는 것입니다. 실제로 Microsoft 또한 HyperGuard로 대응하는 것을 보아 이 점을 인지하고 있는 것 같습니다. 하지만 HyperGuard는 불완전하며 기능이 별로 없다는 것이 알려져 있습니다.

저는 HG에 가치를 두고 있습니다. 앞으로 가상화 기술이 점점 대두됨에 따라 가상화 기술을 사용하는 장치가 많아질 것이고, HG의 사용을 강제할 수 있을 것이라고 믿고 있습니다. 따라서, PG를 HG에 통합하는 것을 권장하는 바입니다.

물론 현재부터 급진적으로 통합을 바라는 것은 아닙니다. 가상화 기술이 본격적으로 수면 위로 상승하는 날은 아직 멀었다고 생각합니다. 하지만 이는 반대로 말하면 심혈을 기울여서 개발할 시간이 남아있다는 것입니다. 이렇듯 HG를 Slow and Steady하게 개발해 나간다면, Windows 보안을 더욱 견고히 할 수 있을 것입니다.

6.2 Something Else

이 외의 제가 사용했던 보완할 수 있는 취약점을 나열해 보겠습니다.

¹⁶<http://cl.wane.im:25819/Wane/kurasagi/blob/master/kurasagi/Module/Barricade.cpp>

- KeBugCheckEx 0x109 파라미터 제거 (및 인라인 블록 제거)
- prefetchnta 명령 제거
- KiMcaDeferredRecoveryService 후 트랩 추가
- 간접 호출 추가
- 필요하지 않은 난독화 제거
- 고정된 상수 (e.g. 0x4C48B4211BBACBEB) 제거, 버그체크 코드처럼 rol, ror 로 난독화
- 성능에 문제되지 않는 선에서 가상화 추가
- KiTimerDispatch 섹션의 W 권한 제거

7 Conclusion

이상으로 PG를 우회하여 살펴본 바와 같이, PatchGuard는 혁신적인 보호 기법으로서 대부분의 리버스 엔지니어들을 혼란에 빠뜨릴 만큼 강력하지만, 숙련된 공격자를 완전히 저지하기에는 여전히 한계가 존재합니다. 따라서 Windows 커널 보안을 한 단계 더 끌어올리기 위해서는 PatchGuard의 잠재적 취약점을 보완하고, 이를 넘어서는 하이퍼바이저 기반 다계층 방어 체계를 도입 및 강화할 필요가 있습니다. 본 고찰이 향후 커널 보안 연구 및 실무적 방어 전략 수립에 작은 길잡이가 되기를 바랍니다. 감사합니다.

7.1 Precautions with Demonstration

전체 코드는 Git 저장소에 있으며¹⁷, 이 코드는 PG 전체를 우회하는 코드입니다. 즉, 드라이버를 로딩하면 PG는 종료됩니다. 해당 코드는 Microsoft Visual Studio 2022로 빌드할 수 있습니다. 이때 WDK 및 Windows SDK가 시스템 내에 설치되어 있어야 빌드가 가능합니다.

시연하실 경우 다음과 같은 주의사항을 숙지해 주세요.

1. 이 PG 우회 코드는 Windows 11 24H2 Build 26100.4351 에서만 작동합니다.
2. 코어 격리 관련 옵션을 모두 끄셔야 합니다.
3. 저는 kdmapper¹⁸를 사용하여 서명되지 않은 드라이버를 시스템 상에 올려 PG 우회를 실험하였습니다. 다만, 일반적인 방법으로 드라이버를 로딩해도 정상적으로 작동합니다.
4. VerifyPg를 통해 NT 함수를 후킹하여 PG를 강제로 트리거할 수 있습니다. kurasagi 드라이버를 로딩할 경우 성공적으로 PG가 우회되어 BSOD가 나지 않는 모습을 볼 수 있습니다.

¹⁷<http://cl.wane.im:25819/Wane/kurasagi>

¹⁸<https://github.com/TheCruZ/kdmapper/tree/master>